

A close-up photograph of a young woman with long, wavy blonde hair. She is smiling and looking off to the right. She is wearing a blue and white horizontally striped sweater. The background is blurred, showing what appears to be an outdoor setting with a brick building and some greenery.

Klasser som værktøjskasser

# Program:

- Hvad er en hjælpeklasse?
- Hvordan laver vi en?
- Hvad er det `import` gør?
- Hvad er et navnerum (namespace)?
- Hvordan laver vi en package?
- Øvelser
- Opsummering

Indtil nu har vi skrevet alt i én fil

Store programmer kræver **organisering**

I et objektorienteret sprog (som Java)  
organiserer vi kode i **klasser**

Konventioner:

- Én klasse per fil
- Filnavn (`Math.java`) = klassenavn (`Math`)



# Hjælpeklasser



Nogle klasser er **værktøjskasser** med nyttige metoder



Eksempel: Hjælpeklassen `java.lang.Math`

```
import java.lang.Math;
```

```
Math.sqrt(16);
```

```
Math.pow(2, 3);
```

```
Math.max(3, 7);
```

Bemærk: Vi skriver altid **Math.** foran metoderne

- det er fordi klassen hedder **Math**

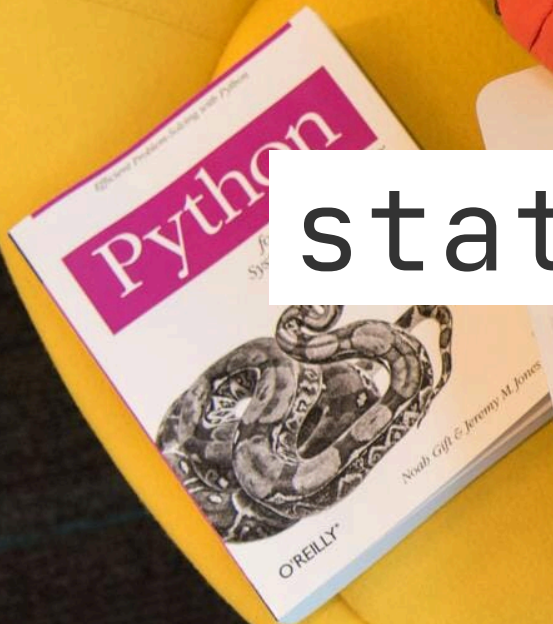
Vi kunne også skrive

```
java.lang.Math.sqrt(16);  
java.lang.Math.pow(2, 3);  
java.lang.Math.max(3, 7);
```

Metoderne i `java.lang.Math` er `static`

DEMO: Er **Math** klassens metoder **static**?

static metoder





Spørgsmål: Hvad betyder **static**?

Svar: Metoden tilhører klassen, ikke et objekt

Dvs. vi behøver ikke at lave et objekt af klassen Math for at bruge metoderne

Med andre ord, vi behøver **ikke**

```
Math math = new Math();  
math.sqrt(16);
```

... vi kan bare skrive

```
Math.sqrt(16);
```

Derfor har alle vores egne metoder, vi har kaldt fra `main`, også været `static`

Fordi vi har kaldt metoder indenfor samme klasse som `main`,  
har vi ikke skullet skrive klassens navn foran

```
public class Main {  
    public static void main(String[] args) {  
        sayHello();  
    }  
  
    public static void sayHello() {  
        System.out.println("Hello!");  
    }  
}
```



Vi kunne også...

```
public class Main {  
    public static void main(String[] args) {  
        Main.sayHello(); // Main. foran  
    }  
  
    public static void sayHello() {  
        System.out.println("Hello!");  
    }  
}
```

Men så er der jo også **String** og **Scanner** og **Random**

Her skulle vi **instantiere et objekt** af klassen først, for at bruge metoderne

a.k.a.

```
String text = new String("Hello");  
Scanner input = new Scanner(System.in);  
Random random = new Random(342);
```

og så kalde metoder på objektet:

```
text.toUpperCase();  
input.nextLine();  
random.nextInt(10);
```

Det vil vi selvfølgelig også kunne med vores egne klasser

Men - det er emnet i en senere i kurset

A young man with short, curly brown hair, wearing a dark blue t-shirt, is seen from the side, looking out of a large glass window. His right hand is resting on the window frame. In the foreground, there are large, green, out-of-focus leaves of a plant, possibly a palm or similar tropical plant. The background outside the window shows a bright, sunny day with some greenery and a blue metal railing visible through the glass. The overall mood is contemplative and bright.

package og import



Vi har brugt **import** til at hente klasser fra Java's standardbibliotek, fx.

```
import java.util.Scanner;
```

Hvad gør `import`?

`import` gør det muligt at bruge klasser fra andre pakker  
... uden at skulle skrive den fulde sti hver gang.

DEMO: Lad os lede efter **Scanner**-klassen i Java's kildekode

I toppen af **Scanner.java** står der:

```
package java.util;
```

Så **Scanner**-klassen tilhører pakken **java.util**. Altså kan vi importere den med:

```
import java.util.Scanner;
```

Konvention: Pakke-navne er altid små bogstaver, fx

```
package calculator;
```

Konvention: Pakke-navne med flere niveauer adskilles med punktum, fx

```
package dk.kea.calculator;
```



Store apps bruger ofte domænenavne bagfra i pakke-navne, fx

```
package com.google.code.gson;  
package org.springframework.boot;
```

Konvention: Pakke-navne og mappenavne skal matche.

Eksempel: `dk.kea.calculator.Math` → mappen  
`dk/kea/calculator/Math.java`

Hvorfor er pakker nyttige?

Pakker fungerer som **navnerum** (namespaces)

Jeg har lavet en klasse der hedder **Math**

Hvad nu hvis Java også havde en klasse der hed **Math**?

**Navnekonflikt!**

... hvis ikke det var for **package**s

så vi kunne kalde min klasse `dk.kea.calculator.Math` ved at:

.. ligge min `Math.java` i mappen `dk/kea.calculator/`



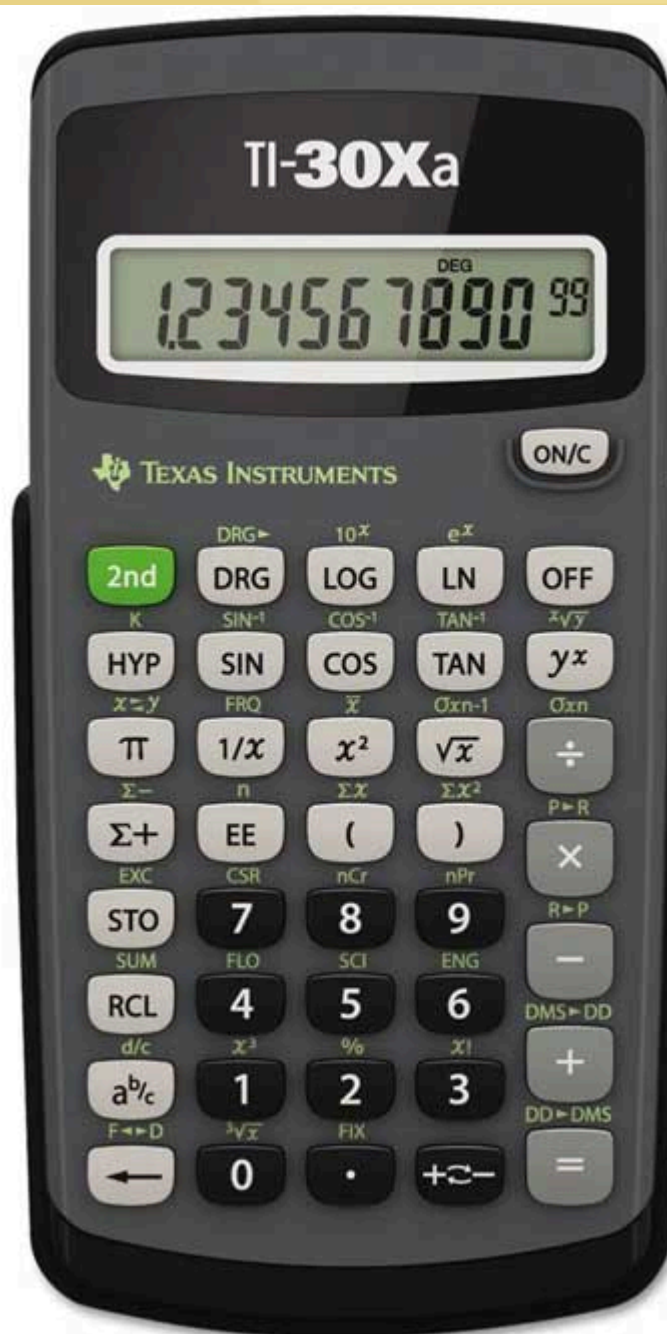
... tilføje flg. i toppen af filen:

```
package dk.kea.calculator;  
public class Math {  
    // ...  
}
```

... og så importere den i en anden fil med:

```
import dk.kea.calculator.Math;
```

DEMO: En lommeregner i pakken dk.kea.calculator



Nævn tre ting du tager med fra i dag?